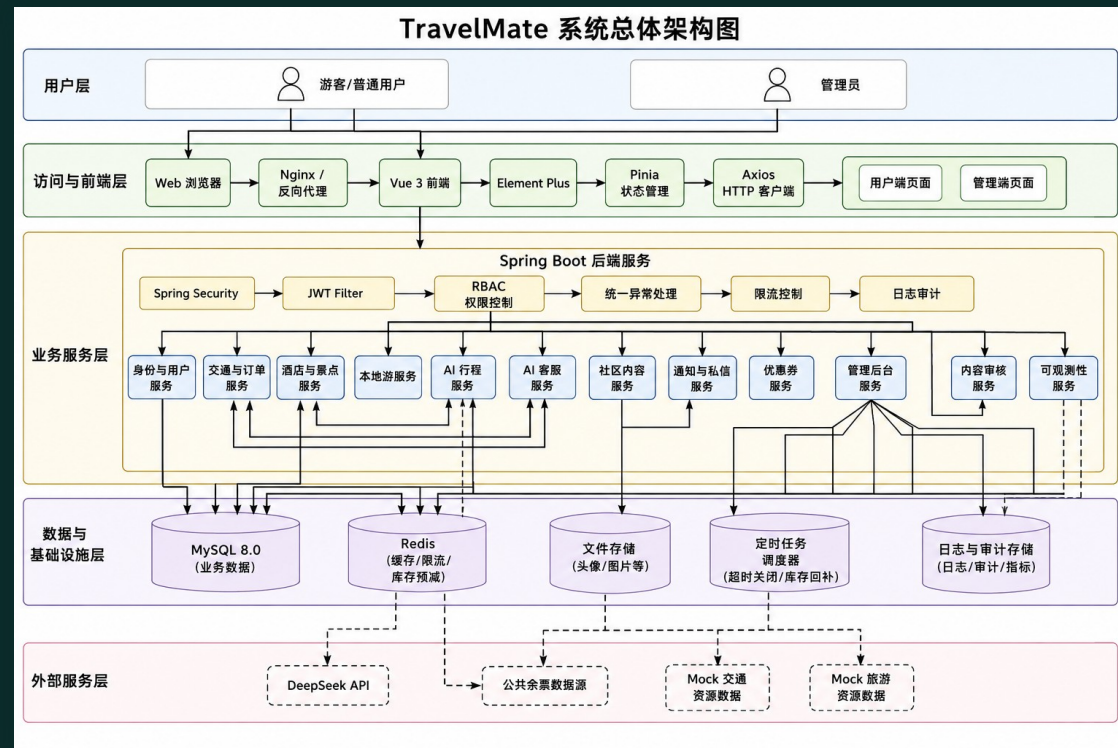


TravelMate 伴游

一站式智慧旅行平台

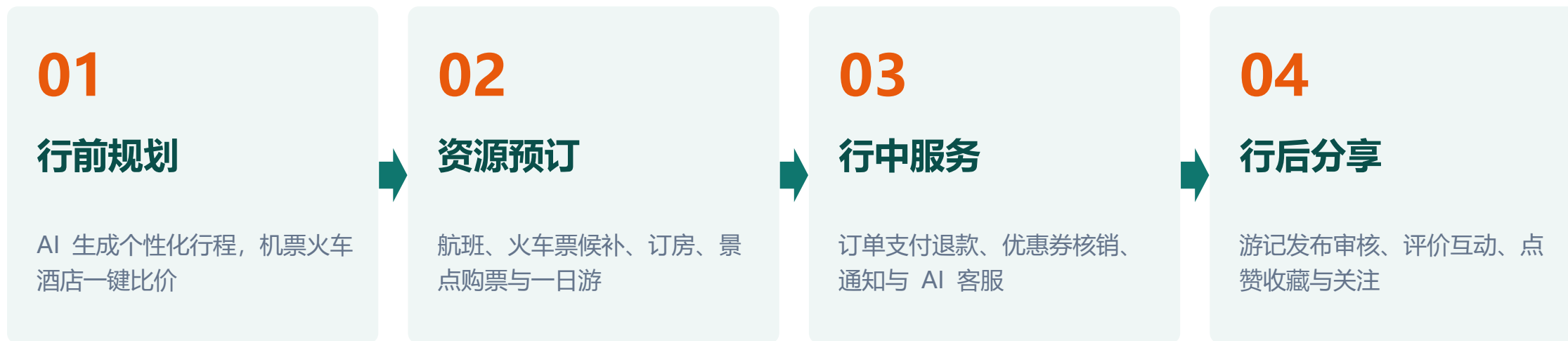
覆盖 " 行前规划 → 资源预订 → 行中服务 → 行后分享 " 全链路，
19 个业务场景全部具备自动化验收证据的完整工程实践。

Miracle 团队 • 2026



一个真实的旅行平台，一次完整的工程实践

TravelMate 不是玩具 Demo —— 它是一个主链路可运行、可部署、可追溯的旅行服务平台：



模拟支付与出票环境，聚焦工程完整性： 需求 → 设计 → 实现 → 测试 → 交付，每一环都有真实产物与流水线证据。

19 个业务场景 · 五大业务域



每个用例均对应系统级、组件级、对象级三层设计模型与自动化测试，追溯覆盖率 100%。

前后端分离 • 渐进式走向微服务

接入层

Vue 3 + Element Plus 单页应用，Nginx 托管，统一 REST 网关

应用层

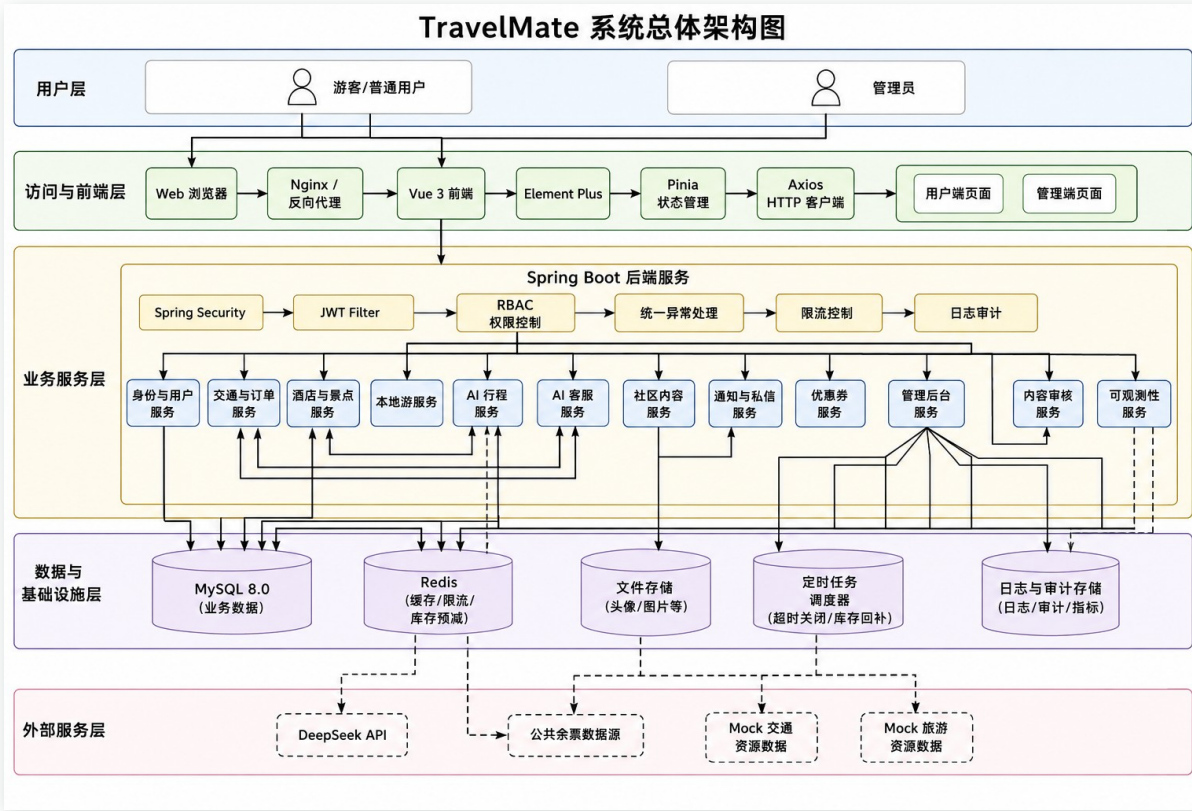
Spring Boot 3.5 单体优先，按领域分包：票务、住宿、AI、社区、运营

数据层

MySQL 8.0（Flyway 版本化迁移）+ Redis（缓存、限流、库存扣减）

演进路线

6 个目标微服务边界已冻结，identity / traffic / local / ai 四服务已落地



现代化技术栈，全流程工程化

前端

Vue 3 • Vite • Element Plus • Pinia • ECharts

组件化 SPA，健康检查内建

全链路可观测

后端

Java 21 • Spring Boot 3.5 • Spring Security • MyBatis-Plus

JWT 认证，模块化领域分包

安全内建

数据

MySQL 8.0 + Flyway • Redis

版本化迁移；缓存 / 限流 / 防超卖

声明式迁移

微服务

Spring Cloud 风格 • Docker Compose • 独立 Maven 模块

identity / traffic / local / ai

边界已冻结

交付

Docker • Kubernetes • GitHub Actions • GHCR

一条流水线从提交走到线上

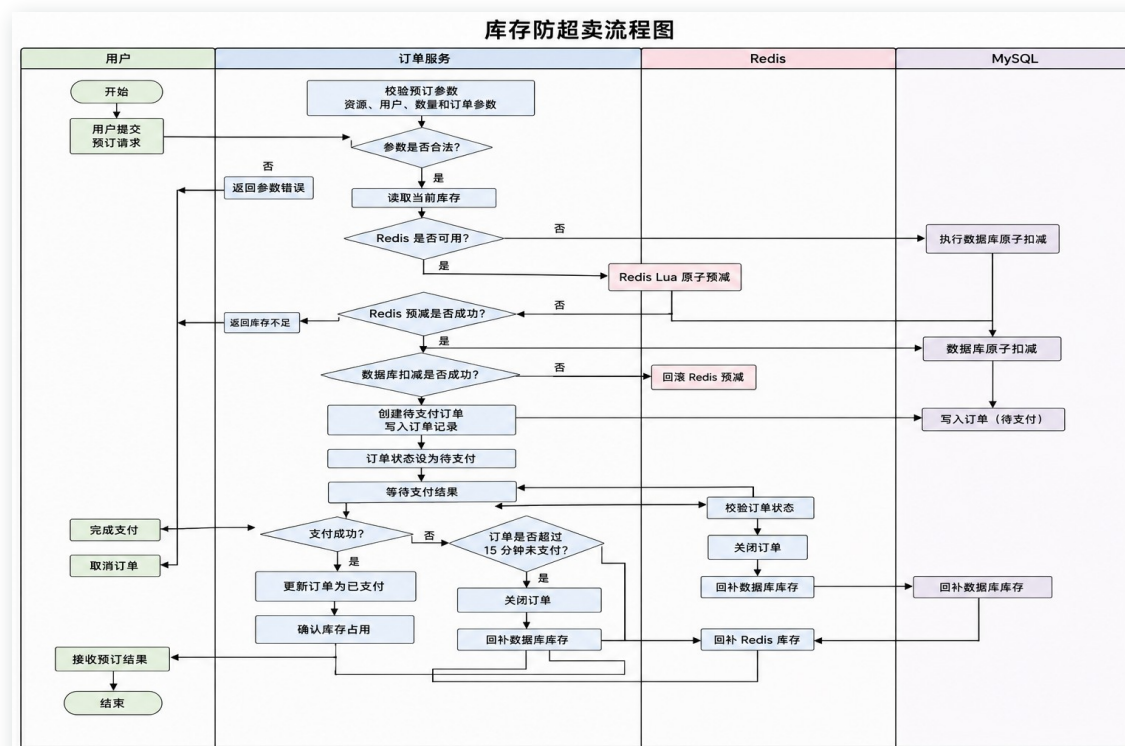
不可变交付

交通票务：高并发下的库存一致性

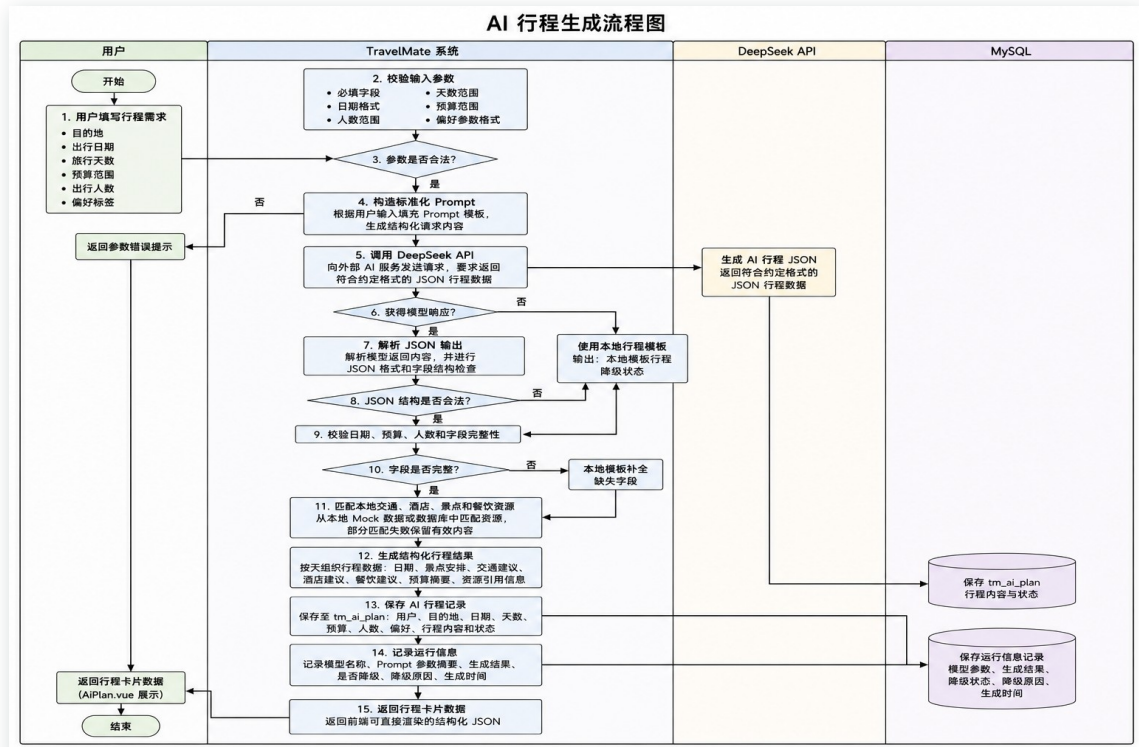
从查票到出票的完整订单状态机

- ▶ 航班实时查询与预订，火车票支持候补队列
- ▶ 订单支付、取消、退款全状态流转，凭证可追溯
- ▶ Redis 预扣库存 + 确认制落库，杜绝超卖
- ▶ 候补兑现、退款回补库存由事件驱动，最终一致

关键设计：库存扣减前置到 Redis 原子操作，数据库只承接确认订单，把热点航段的并发压力挡在数据库之外。



AI 深度融入旅行全流程



会规划的行程助手，随时候命的客服

AI 行程生成（UC11）

输入目的地、天数与偏好，生成可保存、可调整的逐日行程

AI 客服多轮对话（UC12）

订单查询、退改政策、平台使用问题的一站式应答

通知与私信（UC13）

订单状态、候补兑现实时触达；用户间站内私信

社区生态：内容生产与安全并重

游记发布与审核（UC14）

发布、编辑、删除全生命周期，机审 + 人审双通道

评价与举报（UC09）

订单绑定评价防刷分，举报进入处理 workflow

互动与关注（UC15 · 17）

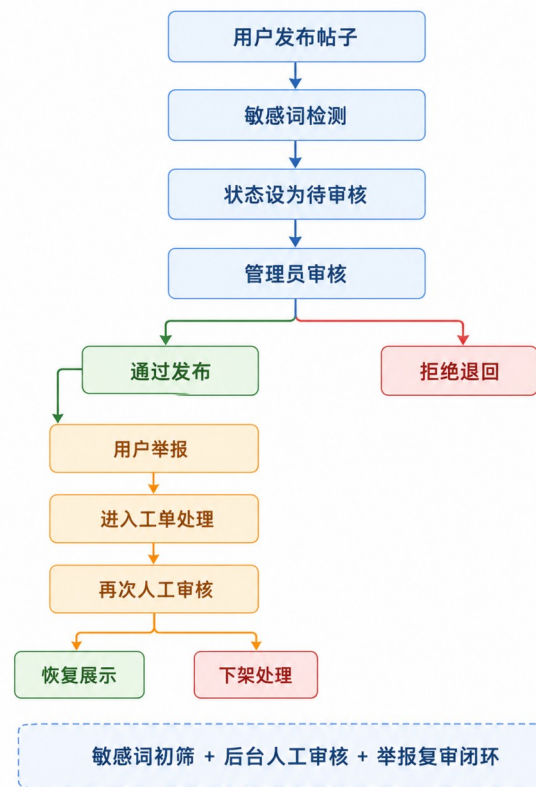
点赞、收藏、评论与关注关系，构建用户主页

内容安全（UC19）

敏感词过滤、审核留痕、全链路可观测性

审核流程：发布 → 机器初审（敏感词）→ 人工复核 → 上架 / 驳回，全程留痕可审计。

社区内容审核流程图





已落地



已落地



已落地



已落地



边界冻结



边界冻结

原则：一表一主，跨服务只能通过接口或领域事件访问数据， Outbox 保证业务与事件同事务。

测试金字塔 + 全链路追溯门禁

161/161

后端单元与集成测试

34 个测试类，真实 MySQL 环境

24/24

前端单元测试

3 个测试文件，流水线自动执行

19/19

业务场景全覆盖

真实后端 E2E，0 partial / 0 planned

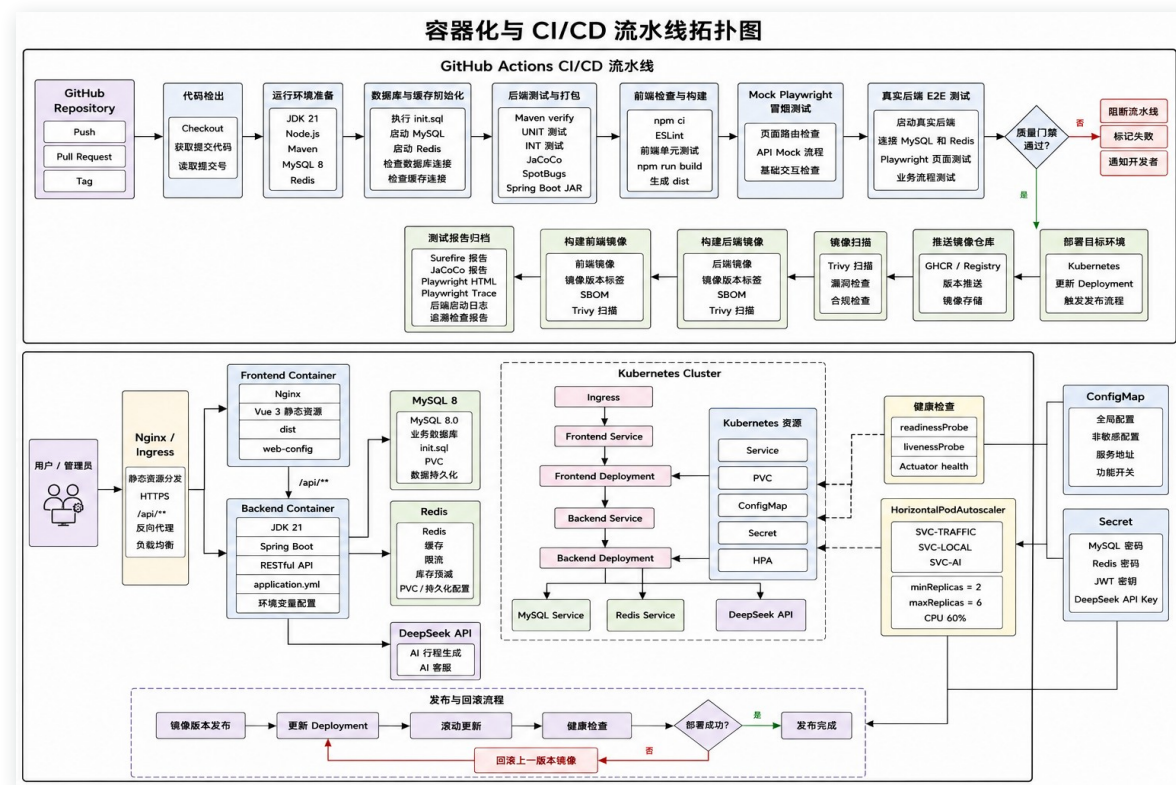
追溯链：每一个需求都能走到测试结果



流水线内置追溯校验脚本：追溯结构或证据不完整，质量门禁直接失败，代码无法合入主分支。

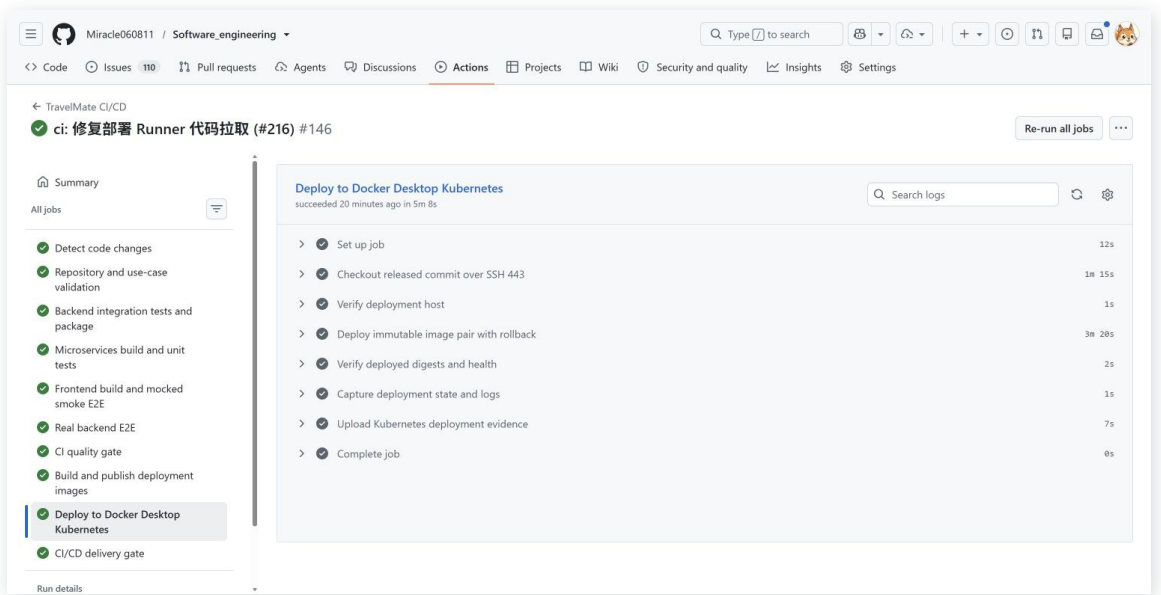
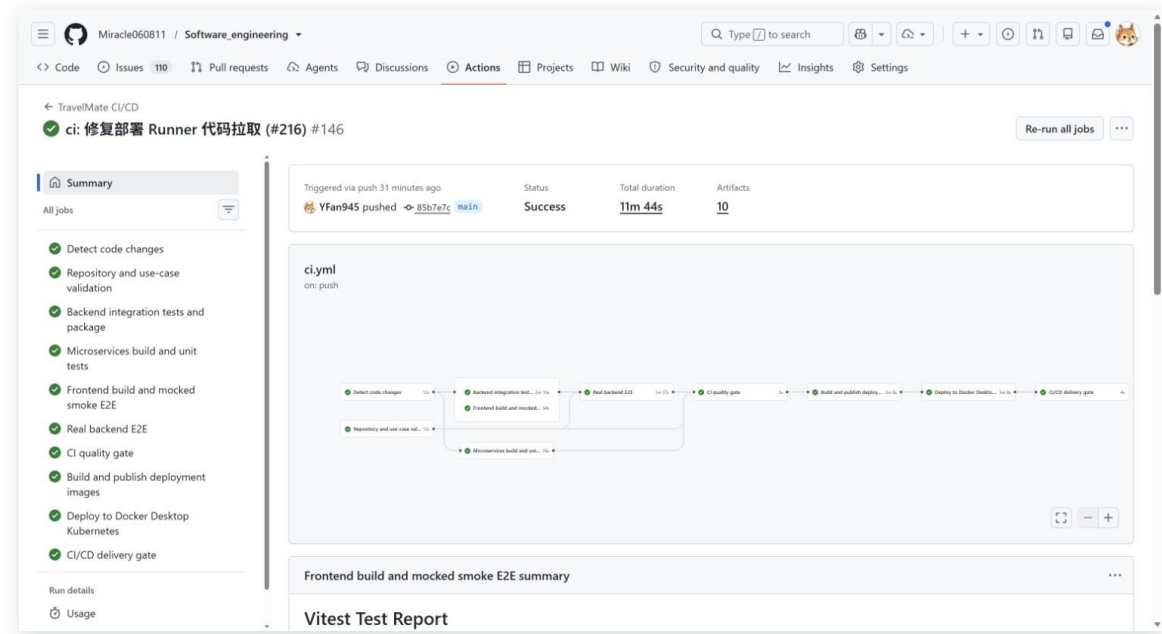
一条流水线，从 git push 到线上健康检查

- 1 配置与追溯校验
- 2 后端 / 前端 / 微服务测试
- 3 真实后端 E2E
- 4 镜像构建 + Trivy 扫描
- 5 GHCR 发布 (不可变 digest)
- 6 K8s 部署 + 健康检查
- 7 交付门禁 delivery gate



任一必需阶段失败，发布与部署即中止；部署使用完整 commit 对应的 digest，绝不使用 latest。

真实运行的成功流水线与线上健康状态



GitHub Actions 全流水线 Success (Run #33154114496 , 总时长 11 分 44 秒)

Kubernetes 部署与健康检查通过: 前后端各 2/2 Ready , 6 个 Pod 全部 Running

证据可复核: 流水线日志、镜像 digest、K8s 运行状态与测试报告共同构成验收证据链。

四道安全门，内建在流水线里

01

Gitleaks

密钥泄露扫描

提交与全量历史扫描，Token、
密码不入库

02

SpotBugs

后端静态分析

字节码级缺陷检测，阻断高危问题
合入

03

Trivy

镜像漏洞扫描

前后端镜像逐层扫描，高危漏洞不
放行

04

CodeQL

代码语义分析

GitHub 官方语义引擎，持续追踪
注入与漏洞模式

安全左移：四道门全部为流水线必需阶段，任何一道不通过，镜像不会发布、应用不会部署。

项目全景数据

19

个业务场景

UC01-UC19 全部自动化验收

185

个自动化测试

后端 161 + 前端 24 , 全部通过

60+

张设计模型图

系统级 / 组件级 / 对象级三层

6

个目标微服务

4 个已落地, 边界全部冻结

100%

追溯覆盖率

需求 → 设计 → 代码 → 测试

11'44"

端到端流水线

提交到 Kubernetes 部署完成

Miracle 团队 · 五人分工



邹林利
组长

大交通票务与订单域



莫谨瑞
组员

住宿与本地生活域



陈一鸿
组员

AI 规划与 Agent



杜新诚
组员

社区与用户画像



李科
组员

管理后台与可观测性

按业务领域纵向分工：每人端到端负责自己领域的需求、设计、实现、测试与交付。

CLOSING

把一条完整的软件工程链路 真正跑通

从 19 个用例到 185 个自动化测试，从单体到六个微服务边界，
从一行提交到 Kubernetes 上的健康检查 —— 每一步都有证据。

github.com/Miracle060811/Software_engineering